

Compile-Time Polymorphism for Runtime-Flexible Systems

Lessons from OpenJDK

SHUBHANKAR GAMBHIR



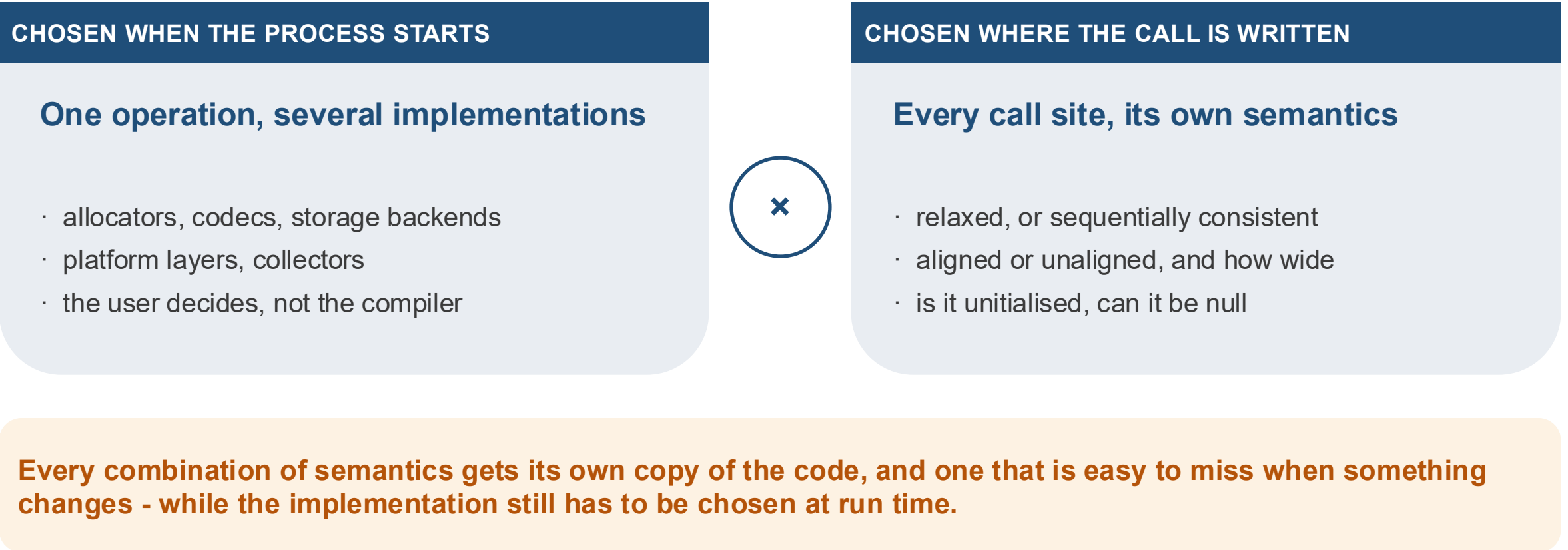
Shubhankar Gambhir

C++ developer making life easier for Java developers

- Software engineer, Azul Systems
- JVM internals and performance
- Systems and low-level code

linkedin.com/in/shubhankar-gambhir

Two choices, made in two different places



Case study: the garbage collection barriers in OpenJDK.

Example of Heap mutation

src/hotspot/share/classfile/javaClasses.cpp

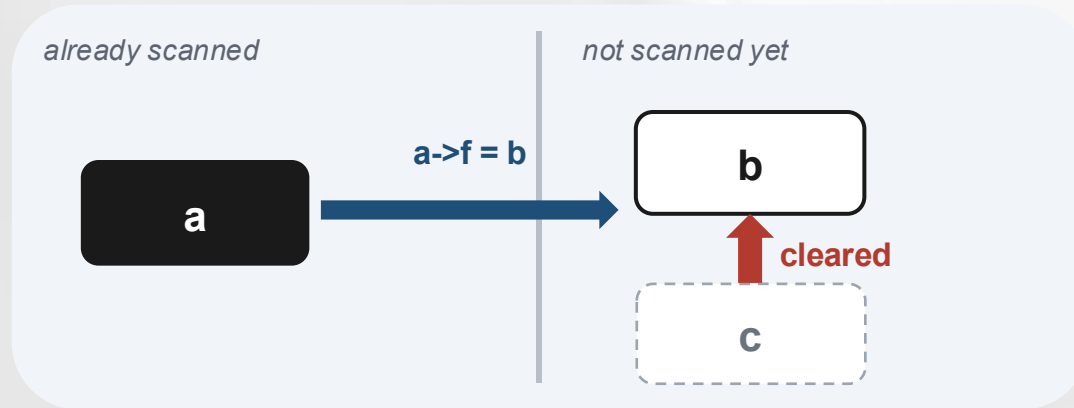
```
void java_lang_Thread::set_name(oop java_thread, oop name) {  
    java_thread->obj_field_put(_name_offset, name);  
}
```

oop is a handle to an object on the collected heap; obj_field_put stores one reference into it.

Setting a thread's name writes one reference field.

Whichever collector is running has to hear about that write, and this code says nothing about which one it is.

A GC barrier is not a memory barrier



1. The collector traces live objects while threads run.
2. A thread writes `b` into `a`, already scanned, and clears the last other pointer to `b`.
3. `b` is never traced, and live memory gets freed.

Nothing to do with ordering: it is bookkeeping for the collector.

What it has to record depends on how that collector traces the heap.

Not the fence you are thinking of

```
// not this: CPU memory ordering
std::atomic_thread_fence(seq_cst);

// this: code the VM emits around
// reference stores on the heap
pre_store(a, &a->f);      // old value
a->f = b;
post_store(a, &a->f, b);  // new value
```


Every collector defines its own barrier

Serial

Store only. Mark a byte in a side table for the region written to.

write barrier: card table

Parallel

The same card table as Serial, marked by many threads at once.

write barrier: card table

G1

Keep the overwritten pointer for the concurrent mark, then log the new one.

write barrier: SATB + remembered sets

Shenandoah

Store barrier plus a check on every heap load, because objects move while threads run.

write + load reference barrier

ZGC

State encoded in unused pointer bits and tested on every load; generational adds store barriers.

load barrier: coloured pointers

Epsilon

Never reclaims anything, so it needs no barrier at all.

no barrier

The nightmare before JEP 304

Six bodies in every binary

The same logic copied at every site

A new collector means editing all of them

```
// Scattered across 100,000s of lines of
// runtime/interpreter code:
if (UseG1GC) {
    // G1's specific tracking logic
} else if (UseShenandoahGC) {
    // Shenandoah tracking logic
} else if ( . . . ) { . . . }
*addr = obj; // The actual memory write
if (UseG1GC || UseParallelGC) {
    // Post-write tracking logic
} else if ( . . . ) { . . . }
```

Pre-JEP 304 code shape · openjdk.org/jeps/304

Nobody can stay in their own lane: runtime work needs every collector's tracking logic, and collector work needs every call site.

How did OpenJDK do it?
How can we solve the
problem with modern C++?

Refactoring (JEP 304)

Goal

- Decouple JVM and GC

Modularity

- Supporting New GC Simpler

Transition

- Imperative -> Declarative

This was the prerequisite for ZGC: a new collector in the same release, with no call sites touched.

Every access carries its semantics with it

Five kinds of semantics travel with every access

Memory ordering

MO_RELAXED, MO_SEQ_CST

Location

IN_HEAP, IN_NATIVE

Barrier strength

AS_NORMAL, AS_RAW, AS_NO_KEEPAIVE

Reference strength

ON_STRONG_REF, ON_WEAK_REF, ON_PHANTOM_REF

Optimization hints

IS_NOT_NULL, IS_DEST_UNINITIALIZED

One set of flags, passed as one value, read by the compiler.

the alternative: one parameter per dimension

```
// oops/accessDecorators.hpp
struct BarrierSet {
    void store(void* addr, T value,
              bool in_heap,
              bool is_not_null,
              int memory_order,
              int barrier_strength,
              int ref_strength,
              ...);
};
```

Three ways to carry the semantics

1. A function argument

```
// ... x every collector
struct BarrierSet {
    void store(
        DecoratorSet decorators, ...)
};
```

The set is read and branched on at every call.

read and branched at run time

2. A method per combination

```
// ... x every collector
struct BarrierSet {
    void store_in_heap(...)
    void store_in_native(...)
    ...
    void store_in_heap_not_null(...)
    void store_in_heap_as_raw(...)
};
```

Written by hand, so the list grows with every new semantic.

one method per combination

3. A template parameter

```
// ... x every collector
struct BarrierSet {
    template <DecoratorSet
              decorators>
    void store(...)
};
```

Written by the compiler, one instantiation per set.

one method for all of them

But the GC is chosen at run time. Can we still use templates?

Attempt 1: semantics on the method

Would win

One method standing in for dozens of shapes, unused paths compiled away.

Prohibited by the standard

Templates are instantiated on demand, so at class-definition time the compiler cannot know how many specialisations exist, and cannot size the vtable.

[temp.mem]/4: a member function template cannot be virtual.

```
struct BarrierSet {  
    template <DecoratorSet ds,  
              typename T>  
    virtual void store(void* addr,  
                      T value);  
    // error: cannot be virtual  
};  
  
auto* bs = barrier_set();  
    // run-time choice: fine  
bs->store<IN_HEAP | MO_RELAXED>(f, v);  
    // compile-time semantics: not allowed
```

Attempt 2: semantics on the class

Wins

The compiler checks the semantics: one instantiation per combination, nothing written by hand.

Still stuck

Each decorator set is an unrelated class with its own vtable, so one run-time object cannot serve them all.

Templates give compile-time semantics; virtual dispatch gives the run-time collector. Neither attempt gives both.

```
template <DecoratorSet ds>
struct BarrierSet {
    virtual void store(void* addr,
                       oop value) = 0;
};

// every set yields an unrelated type
BarrierSet<IN_HEAP|MO_RELAXED>* a
BarrierSet<IN_NATIVE|MO_SEQ_CST>* b
// different type!
```

The route: from the collectors back up to the JVM

BACK END
the collectors

`BarrierSet::AccessBarrier`

G1 · Z · Serial · Parallel



THE TEMPLATE PIPELINE, IN THE ORDER WE WILL WALK IT

1 The run-time bridge

`RuntimeDispatch`: resolve once, then a pointer

2 Compile-time short-circuiting

`PreRuntimeDispatch`: Raw or dispatch

3 Normalization, injection, reduction

`std::decay_t`, `DecoratorFixup`, `store_reduce_types`



FRONT END
the JVM

`Access::store` · `Access::oop_store_at`

states intent, names no collector

RuntimeDispatch: one function pointer per access kind

One dispatch table per access kind

`func_t`

the shape of a store barrier

`_store_func`

the barrier we call today

`store_init`

what the pointer starts out aimed at

every store goes through the pointer

Resolution is idempotent: two threads racing here write the same pointer.

```
// oops/accessBackend.hpp
template <DecoratorSet ds, ... >
struct RuntimeDispatch {
    typedef ... func_t;           // 1
    static func_t _store_func;    // 2
    static void store_init(...);  // 3

    static void store(...) {
        _store_func(...);
    }
};

// the pointer starts at the resolver
template <DecoratorSet ds, typename T>
func_t RuntimeDispatch<...>::_store_func
    = &store_init;
```

Runtime dispatch: resolve once, dispatch forever

The first store pays; the rest jump

first store at this site



resolve_barrier()



cache it in _store_func



run this store

every later store: straight through the pointer

Resolution happens once per call site, not once per store.

1. The trigger: resolve, cache, store

```
// first store at this site
void RuntimeDispatch<ds,...>::
  store_init(void* addr, T value) {
    func_t f = BarrierResolver<ds,...>
               ::resolve_barrier();
    _store_func = f;    // cache it
    f(addr, value);    // store now
  }
```

2. The resolution: which collector is running

```
// which GC is running in this JVM?
auto* bs = BarrierSet::barrier_set();
switch (bs->kind()) {
  case G1BarrierSet: return G1BarrierSet::
    AccessBarrier<ds>::oop_store;
  case ZBarrierSet: return ZBarrierSet::
    AccessBarrier<ds>::oop_store;
  // one case per GC, macro-generated
}
```

Object references come in two widths

oop

ordinary object pointer

64 bits

8-byte aligned

no encoding on access

larger heap footprint

narrowOop

compressed reference

32 bits

saved

shift and add to decode

encode on store, decode on load

half the footprint

same reference, two representations

Which form an access uses is carried by a decorator, not a separate code path.

The Backend

`BarrierSet::AccessBarrier` · where the collectors plug in

The base class: every access starts out raw

RawAccessBarrier already does the work

Pointer compression

narrow or wide, chosen by decorator

Memory ordering

relaxed, acquire/release, volatile

The write itself

the plain machine store

```
AccessBarrier : protected RawAccessBarrier<ds>
```

A collector overrides only what it needs; the rest stays raw.

```
class BarrierSet {
  template <DecoratorSet ds>
  class AccessBarrier: protected
    RawAccessBarrier<ds> {
    typedef RawAccessBarrier<ds> Raw;

  public:
    // primitive writes need no GC tracking
    template <typename T>
    static void store_in_heap(T* addr,
                              T value) {
      Raw::store(addr, value);
      // fully inlined delegation
    }
  };
}
```

ModRef fixes the order, but the hooks need an object

A fixed skeleton, two holes for the collector

`write_ref_field_pre` (instance method)



`Raw::oop_store(addr, value)`



`write_ref_field_post` (instance method)

ModRefBarrierSet owns the order; the collector fills the hooks.

the entry point is static: there is no this

Template dispatch arrives with a type, not an object.

```
// what we would like to write
template <DecoratorSet ds>
static void oop_store_in_heap(T* addr,
                              oop v) {
    write_ref_field_pre<ds>(addr);
    Raw::oop_store(addr, v);
    write_ref_field_post<ds>(addr);
}
```

```
error: cannot call member function
      'write_ref_field_pre' without
      an object
```

Decoupled CRTP: type from the template, object from the global

The type and the object arrive separately

1

Abandon 'this'

BarrierSet::barrier_set() gives the object

2

Cast with the CRTP parameter

barrier_set_cast<BarrierSetT>(global_bs)

3

Call the hooks on it

bs->write_ref_field_pre<ds>(addr)

Static entry point, instance call, nothing virtual.

```
// gc/shared/modRefBarrierSet.inline.hpp
template <DecoratorSet ds,
          typename BarrierSetT>
inline void ModRefBarrierSet::
  AccessBarrier<ds, BarrierSetT>::
  oop_store_in_heap(T* addr, oop v) {

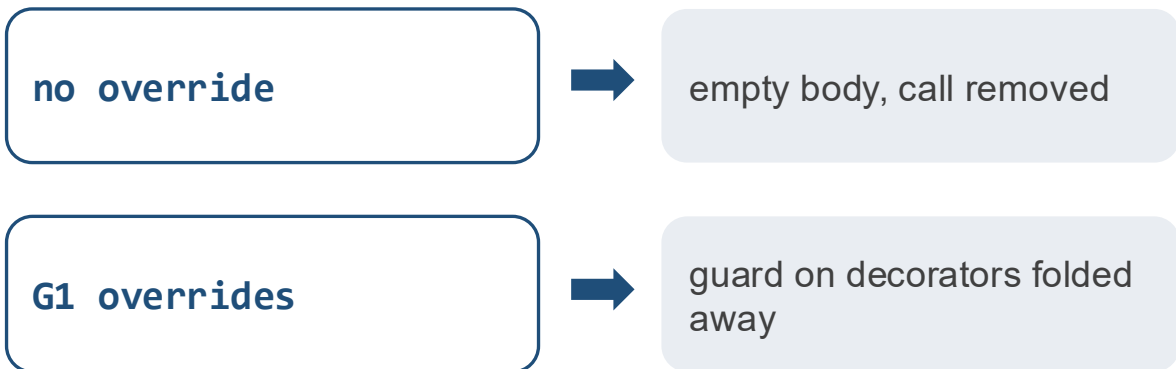
  // 1. abandon 'this', take the global
  BarrierSet* global_bs =
    BarrierSet::barrier_set();

  // 2. cast with the CRTP parameter
  BarrierSetT* bs =
    barrier_set_cast<BarrierSetT>(global_bs);

  // 3. statically bound instance calls
  bs->write_ref_field_pre<ds>(addr);
  Raw::oop_store(addr, v);
  bs->write_ref_field_post<ds>(addr);
}
```

Extension points that cost nothing when unused

What an unused hook costs



the hook you do not use is not in the binary

Decorators are compile-time constants, so the guards are resolved before the call site exists.

Serial / ParalleGC -> Cardtable -> Modref:
pre hook do nothing

```
template <DecoratorSet ds, typename T>
inline void write_ref_field_pre(T* addr) {}
```

G1 specialises, and prunes its own guard

```
template <DecoratorSet ds, typename T>
inline void G1BarrierSet::
write_ref_field_pre(T* field) {
    if (HasDecorator<ds,
        IS_DEST_UNINITIALIZED>::value ||
        HasDecorator<ds, AS_NO_KEEPALIVE>::value)
        return;

    // G1's pre-write barrier
}
```


Downcasting to the concrete BarrierSet, without RTTI

Two casts, and HotSpot can use neither as-is

`dynamic_cast`

needs RTTI and a run-time lookup

HotSpot compiles with `-fno-rtti`

`static_cast`

free, but unchecked on its own

`typeid` would need RTTI too

`static_cast` + the FakeRtti tag check

The tag set carries type identity, so the cast can be verified without RTTI.

```
// 1. dynamic_cast: needs RTTI
BarrierSetT* bs =
    dynamic_cast<BarrierSetT>(global_bs);

// 2. static_cast + typeid: also RTTI
assert(typeid(*global_bs) ==
        typeid(BarrierSetT));

// 3. what HotSpot does
assert(global_bs->is_a(
    BarrierSet::GetName<BarrierSetT>::value));
BarrierSetT* bs =
    static_cast<BarrierSetT*>(global_bs);
```

Type Identity Without RTTI

FakeRttiSupport · a tag set instead of a vtable

FakeRTTI: type identity kept as data

Two fields carry the whole type

`_tag_set`

one bit per class it is-a

`_concrete_tag`

the leaf class it actually is

`has_tag: _tag_set & (1 << tag)`

One AND answers is-a. One uintx of tags caps a hierarchy at 64 classes.

utilities/fakeRttiSupport.hpp

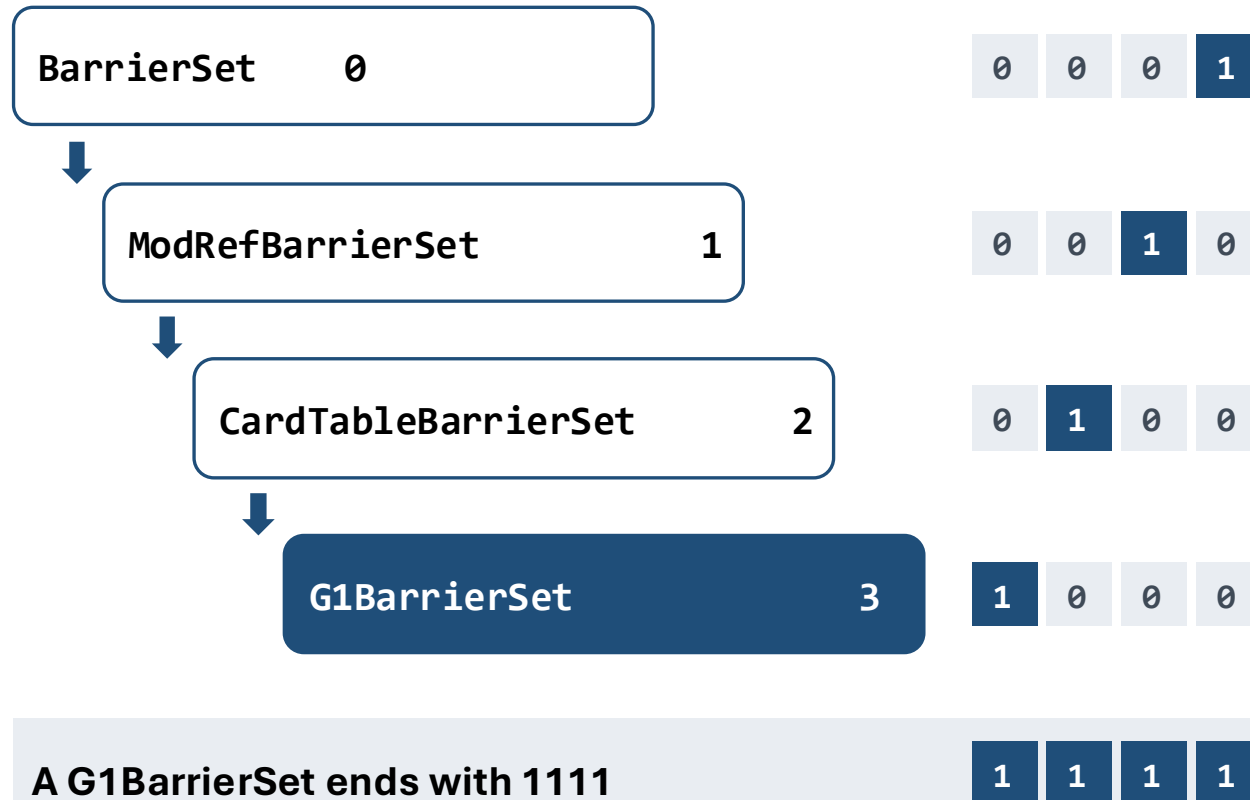
```
// TagType is an enum: one tag per class
template <typename TagType>
class FakeRttiSupport {
    uintx _tag_set; // one bit per class
    TagType _concrete_tag; // the leaf class

    bool has_tag(TagType tag) const {
        return (_tag_set & (uintx(1) << tag))
            != 0;
    }
};

// BarrierSet exposes it as is_a
bool is_a(Name n) const
{ return _fake_rtti.has_tag(n); }
```

FakeRTTI inside the class

Each constructor ORs in its own bit



`is_a`

G1 : tag set	1	1	1	1
ModRef: concrete tag	0	0	1	0
= nonzero	0	0	1	0

Nonzero means is-a: a G1BarrierSet is-a ModRefBarrierSet.

The checked downcast

```
assert(bs->is_a(GetName<T>::value));  
return static_cast<T*>(bs);
```

How a class joins the fake RTTI scheme

1

Declare a Name

one enum entry for your class in the hierarchy

2

Specialise GetName<T>

so the cast can ask the type for its tag

3

Intermediate class: take a FakeRtti

add their Name to the set, pass it up

4

Concrete class: create one

seed the set with their Name, pass it up

```
// barrierSet.hpp
enum Name { ModRef, CardTableBarrierSet,
            G1BarrierSet, ... };

template<> struct GetName<G1BarrierSet> {
    static const Name value = G1BarrierSet;
};

// concrete: seed the tag set
G1BarrierSet::G1BarrierSet(...)
    : CardTableBarrierSet(FakeRtti(
        BarrierSet::G1BarrierSet)) { }

// base: augment, then pass it up
CardTableBarrierSet::CardTableBarrierSet(
    FakeRtti fake_rtti)
    : ModRefBarrierSet(fake_rtti.add_tag(
        BarrierSet::CardTableBarrierSet)) { }
```

PreRuntimeDispatch: when store doesn't need GC

Semantics decide whether the GC is involved

`store<decorators>(addr, value)`



`INTERNAL_VALUE_IS_OOP && !AS_RAW`

↓ No

Raw::store

primitive, or AS_RAW:
plain memory store

↓ Yes

RuntimeDispatch::store

oop that still needs the
barrier

How the choice is expressed

```
// an oop, and not a raw access?
template <DecoratorSet ds>
using NeedsGC = BoolConstant<
    HasDecorator<ds, INTERNAL_VALUE_IS_OOP>
    && !HasDecorator<ds, AS_RAW>>;

template <DecoratorSet ds, typename T>
EnableIf<NeedsGC<ds>::value>::type
store(void* a, T v)
    { RuntimeDispatch<ds, T>::store(a, v); }

template <DecoratorSet ds, typename T>
EnableIf<!NeedsGC<ds>::value>::type
store(void* a, T v)
    { Raw::store(a, v); }

// the other overload never enters the
// candidate set
```

SFINAE

if constexpr

concepts

The front end: callers state intent, not mechanism

The JVM says what it wants, not how

Access::store

a primitive field or array
element

Access::oop_store_at

a reference field, at an offset in
an object



AccessInternal::store / store_at<decorators>

Both calls carry only a decorator set forward.

front end: the JVM · back end: the collectors

Neither side knows the other exists.

```
// oops/access.hpp
class Access: public AllStatic {
  // 1. primitive access
  template <DecoratorSet ds,
            typename P, typename T>
  static void store(P* addr, T v) {
    verify_primitive_decorators<ds>();
    AccessInternal::store<ds>(addr, v);
  }

  // 2. object field access
  template <DecoratorSet ds, typename T>
  static void oop_store_at(oop base,
                          ptrdiff_t off, T v) {
    verify_heap_oop_decorators<ds>();
    AccessInternal::store_at<
      ds | INTERNAL_VALUE_IS_OOP>(
      base, off, v);
  }
};
```

Type normalization: dozens of types become a handful

Many written types, few compiled ones

`const int*, volatile int*`



`int*`

`jobject, instanceOop,
objArrayOop`



`oop`

`narrowOop`



`narrowOop`

`narrowOop` is the one type kept apart, so the compressed path stays reachable.

The qualifiers are not lost: `volatile` returns as a decorator.

```
// qualifiers off
using DecayedP = std::decay_t<P>;
using DecayedT = std::decay_t<T>;

// every oop type becomes oop ...
template <typename T>
struct OopOrNarrowOopInternal {
    typedef oop type;
};

// ... except narrowOop, which is kept
template <>
struct OopOrNarrowOopInternal<narrowOop> {
    typedef narrowOop type;
};

// forward the clean types
store_reduce_types<ds>(
    const_cast<DecayedP*>(addr), decayed);
```


Decorator injection: the set is fixed up before dispatch

What normalization dropped comes back

volatile field → MO_RELAXED

oop field access → INTERNAL_VALUE_IS_OOP

value is an oop → INTERNAL_CONVERT_COMPRESSED_OOP

PreRuntimeDispatch::store_at<expanded>

Each phase only adds to the set; nothing is re-derived later.

```
// 1. the front end flagged the oop:
//   store_at<ds | INTERNAL_VALUE_IS_OOP>

// 2. volatility comes back as ordering
DecoratorSet ds2 = DecoratorFixup<
  (std::is_volatile<P>::value
   && !HasDecorator<ds, MO_DECORATOR_MASK>)
  ? (MO_RELAXED | ds) : ds>::value;

// 3. an oop value may need compressing
DecoratorSet ds3 = DecoratorFixup<ds2 |
  (HasDecorator<ds2, INTERNAL_VALUE_IS_OOP>
   ? INTERNAL_CONVERT_COMPRESSED_OOP
   : DECORATORS_NONE)>::value;

// on to phase 2
PreRuntimeDispatch::store_at<ds3>(
  base, offset, decayed_value);
```

Type reduction: the pairing decides the compression

Value type T	Address type P	What it means
T (primitive)	T*	No compression, types reduced
oop	oop*	No compression
narrowOop	narrowOop*	Compression, confirmed
oop	narrowOop*	Compression, confirmed
oop	HeapWord*	Compression, ambiguous

Every compressed case gets `INTERNAL_CONVERT_COMPRESSED_OOP`. The confirmed ones also get `INTERNAL_RT_USE_COMPRESSED_OOPS`.

Only these five pairings exist, and each one settles the question before dispatch.

Three overloads, one per outcome

No compression

T* with a T

```
template <DecoratorSet decorators, typename T>
inline void store_reduce_types(T* addr, T value) {
    PreRuntimeDispatch::store<decorators>(addr, value);
}
```

Confirmed compression

narrowOop* with either

```
template <DecoratorSet decorators>
inline void store_reduce_types(narrowOop* addr, oop value) {
    const DecoratorSet expanded = decorators |
        INTERNAL_CONVERT_COMPRESSED_OOP | INTERNAL_RT_USE_COMPRESSED_OOPS;
    PreRuntimeDispatch::store<expanded>(addr, value);
}
// same body again for (narrowOop* addr, narrowOop value)
```

Ambiguous compression

HeapWord* with an oop

```
template <DecoratorSet decorators>
inline void store_reduce_types(HeapWord* addr, oop value) {
    const DecoratorSet expanded = decorators | INTERNAL_CONVERT_COMPRESSED_OOP;
    PreRuntimeDispatch::store<expanded>(addr, value);
}
```

Trade-offs and Takeaways

JEP 304 · what it bought, and what it cost

One field assignment, before and after JEP 304

THE CALL SITE IN THE JVM, UNCHANGED

```
java_thread->set_name(name);           // one reference field assignment
```

What it was · JDK 8 to 10

```
// obj_field_put branches on
// UseCompressedOops, then:
template <class T> void oop_store(T* p, oop v) {
    if (RuntimeFlag)
        return oop_store((volatile T*)p, v);

    update_barrier_set_pre(p, v);      // virtual
    encode_store_heap_oop(p, v);
    update_barrier_set(p, v, false);  // virtual
}
```

2 flag branches → 0

UseCompressedOops and the CMS flag are compile-time constants now

2 virtual calls → 1 indirect call

one function pointer, resolved on the first store at each site

What it is now · JEP 304

```
// one call, semantics as decorators
HeapAccess<>::oop_store_at(as_oop(), off, v);

// what the compiler keeps:
//   no flag branches
//   one resolved function pointer
//   the collector's barrier, inlined

// a primitive skips the barrier path
RawAccess<>::store(addr, value);
```

unused barrier → nothing

an empty hook inlines away and the call site disappears

JEP 304 set out to remove duplicated code without regressing performance; the speed comes from what the compiler can now delete.

What the mechanisms cost, roughly

Approach	ns/call	Why
Direct call (baseline)	1.44	Target known when compiling
Function pointer	2.39	One indirect call, no composition
Decoupled CRTP (lazy)	2.39	One indirect call, composable
Virtual dispatch	2.87	Two dependent loads
<code>std::variant + std::visit</code>	3.72	Closed set, stdlib-dependent

Xeon Gold 6130 · 100M iterations · dispatch mechanism only, barrier body excluded · GCC 11, -O2

GCC 12 added a switch optimisation to `std::visit` that drops variant to 1.44 ns. The ranking is a property of your toolchain, not of C++.

40 - 70 ns

once per call site, the first time through: a cold indirect branch plus the resolve

Every later store at that site is a plain indirect call - the resolution amortises to nothing.

These come from my own micro-benchmark of the mechanisms, not from OpenJDK's implementation. Take them as orders of magnitude, and measure your own case.

Modularity: four pieces, changed independently

1

Abstract base classes

BarrierSet::AccessBarrier over RawAccessBarrier - the default for every operation is a plain store

2

Specialised implementations

ModRef fixes the order of the hooks; G1 fills them in and prunes the ones its decorators make pointless

3

Run-time setup

the JVM builds exactly one barrier set at startup, from the flag it was given

4

Resolution after startup

the first store at each call site patches in the resolved barrier; every later one is a plain indirect call

A new collector touches piece two. A new kind of access touches piece one. Neither has to know the other exists.

Where this design pays, and where it does not

Worth paying for

You cannot skip the barrier

The only path through the API is the correct one. In a code base this size that is worth more than the nanoseconds.

Run-time cost at parity

One indirect call once the site is resolved - the same as a direct function pointer.

What it costs

Build times

Every decorator combination is another instantiation. This is a real cost in OpenJDK, not a theoretical one.

Five frames deep in a stack trace

Stepping through the pipeline in a debugger at three in the morning is the cost nobody puts in the JEP.

Worth it where a wrong call site is expensive. Otherwise a function pointer buys most of this for none of the complexity.

Four steps to steal from this

1

Identify what is already known when you compile

The semantics were at the call site all along

2

Separate the axes that vary

The implementation choice is genuinely run-time. The semantics are not. Do not let one drag the other down.

3

Move the compile-time axis into the type system

A template parameter, a constexpr predicate, or a concept - whichever your dialect allows.

4

Then simplify

Constant conditions make whole paths dead, and the compiler deletes them. The generated code gets smaller, not bigger.

Any system where the user picks an implementation at startup, and you cannot pay for that choice on every call.

Resources

OpenJDK sources

- `access.hpp` the public API
- `accessBackend.hpp` the trampoline
- `barrierSet.hpp` the CRTP seam
- `barrierSetConfig.hpp` GC macro list
- `g1BarrierSet.inline.hpp` G1
- github.com/openjdk/jdk

Background

- JEP 304 — openjdk.org/jeps/304
- JBS issue — [JDK-8163329](https://bugs.openjdk.org/browse/JDK-8163329)
- Design thread — [hotspot-gc-dev, Apr 2017](https://mail.openjdk.org/list/commit-2017-04-04.html)

Longer write-ups

- shubhankar-gambhir.github.io
- *Four Ways to Dispatch a Runtime-Selected Strategy*
- *Lazy Resolution: Resolve Once, Dispatch Forever*
- *Why std::visit May Be Slower Than a Vtable*
- *Your Stdlib Matters More Than the Pattern*

C++ patterns & further reading

- Coplien (1995) — the original CRTP write-up
- Barton & Nackman — the base-class trick
- Sean Parent — *Inheritance Is the Base Class of Evil*
- Klaus Iglberger — CppCon type-erasure talks
- Louis Dionne — Dyno, policy-based dispatch
- Shipilëv — shipilev.net/jvm/diy-gc/

Q & A

Thank You